

Recall Does Not Move at a Trillion Rows [Scalable Data Science]

Routed search over compressed vectors, measured against the exact scan, on a shared research cluster

Andrew H. Bond
San José State University
San José, California, USA
andrew.bond@sjsu.edu

ABSTRACT

We report a measurement of routed inverted-file search over an index of one trillion 4-bit quantized vectors, built and queried on the National Research Platform’s shared Kubernetes cluster. The index is 500 servers of two billion rows each, 24.0 bytes per row, 24 TB in all, on one block volume per server. Against the exact full scan of the same compressed index, routed search returns recall at ten of 0.989 when it probes 32 of 2048 cells and 0.999 when it probes 128, on 100 queries, with 95 percent bootstrap intervals of 0.982 to 0.995 and 0.997 to 1.000. The same protocol gave 0.979 and 0.999 at one hundred million rows, 0.982 and 0.999 at ten billion, and 0.983 and 0.999 at one hundred billion. Recall against the full scan is stable across four orders of magnitude of corpus size. Every scale point draws its queries by the same recipe, the first rows of four seeded shards spread across the index, so the comparison across sizes holds the query distribution fixed. That recipe also places the reference neighbours on few servers, and a query set drawn from many shards is not measured here. The second result is that this recall is a property of the router alone. From the cell of each reference neighbour in its query’s probe order we predicted recall at nine probe widths without a scan, the prediction matched the two widths already measured to the last digit, and a sweep at three further widths, run after the prediction was recorded, returned 0.952, 0.997 and 1.000, each equal to its predicted value. The measure is self-consistency of the compressed index on a synthetic corpus of intrinsic dimension 16, and the paper gives the float ground truth and real-embedding figures that bound it. Reranking each shortlist against float rows regenerated from the seed shows that the compressed ranking keeps at most 0.621 of the true neighbours on this corpus, and after that rerank the routed top ten agrees with the reranked reference at 0.982 for 32 probes and 1.000 from 128 on. The third contribution is the measurement design. Every job runs in one CPU and two gigabytes of memory, the smallest class the cluster offers, and a pool of idempotent per-server jobs submitted through a controller that yields when the cluster is busy carried 1502 submissions to 1350 completions over four days. Every log, state file, partial result and score is in the repository, and the two identities the analysis rests on are checked in Lean 4 with Mathlib.

PVLDB Reference Format:

Andrew H. Bond. Recall Does Not Move at a Trillion Rows [Scalable Data Science]. PVLDB, 20(1): XXX-XXX, 2027.

doi:XX.XX/XXX.XX

This work is licensed under the Creative Commons BY-NC-ND 4.0 International License. Visit <https://creativecommons.org/licenses/by-nc-nd/4.0/> to view a copy of this license. For any use beyond those covered by this license, obtain permission by emailing info@vldb.org. Copyright is held by the owner/author(s). Publication rights licensed to the VLDB Endowment.

Proceedings of the VLDB Endowment, Vol. 20, No. 1 ISSN 2150-8097.

PVLDB Artifact Availability:

The source code, data, and/or other artifacts have been made available at <https://github.com/ahb-sjsu/turboquant-pro/tree/master/benchmarks/fleet>.

1 INTRODUCTION

Approximate nearest-neighbour search over compressed vectors is routine at a billion rows [2, 8, 10, 16]. The question this paper answers is narrower than most. When an inverted-file index is partitioned into shards that share one coarse quantizer and one rotation, and the corpus grows by adding shards rather than by densifying old ones, does the routing lose anything it would have found by scanning everything? The way to ask it is to build the whole index, scan the whole index exactly, and compare. We did that at one trillion rows.

The index was built and measured on the National Research Platform’s Nautilus cluster [18], a Kubernetes federation shared by many research groups across sites. A fleet measurement there is a matter of fair use by design. Jobs are small, independent and resumable, submissions are paced by a controller that yields when the cluster is busy, and nothing is reserved. The second half of the paper describes that design and what it did over four days.

The paper makes three claims and states one limit. Routed recall against the exact scan of the same index is 0.989 at 32 probes and 0.999 at 128 probes at 10^{12} rows, with 95 percent bootstrap intervals over the 100 queries of 0.982 to 0.995 and 0.997 to 1.000, and the same protocol at 10^8 , 10^{10} and 10^{11} rows gave figures within 0.01 of these at 32 probes and within 0.001 at 128 probes (Section 4). The queries at every scale point are the first rows of four seeded shards spread across the index, so the trend across sizes is measured with the query recipe fixed, and Section 4.3 states what that recipe does to the placement of the reference neighbours. That recall is the share of reference neighbours whose cell the router reaches, which was computed from the index metadata alone, matched the measured widths, and predicted three further widths before they were measured (Section 4.2). Every phase of the measurement runs in one CPU and two gigabytes of memory, and a pool of idempotent jobs with a small number of explicit rules completes a 1502-submission run on a shared cluster (Sections 5 and 6). The limit is that the corpus is synthetic, of intrinsic dimension about 16, and the recall reported is against the compressed index’s own exact scan rather than against float ground truth. Section 7 gives the figures that bound the result, including a real-embedding pilot on which the routing holds and the compression alone does not.

doi:XX.XX/XXX.XX

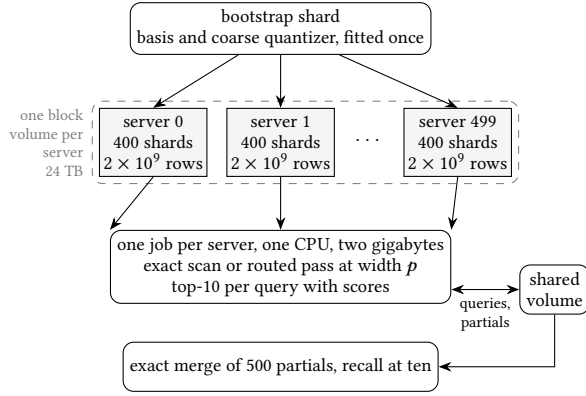


Figure 1: The fleet. One bootstrap shard fixes the projection basis and the 2048-cell coarse quantizer for every shard on every server, so scores are comparable across servers and the global top-10 is the exact merge of per-server top-10s. Each phase is one job per server, and each job reads only its own volume and the shared volume.

2 THE INDEX AND THE CORPUS

Index. Each row is a 32-dimensional vector projected to 24 dimensions by one basis shared by the whole fleet, quantized to 4 bits per dimension, and assigned to one of 2048 inverted-file cells by one shared coarse quantizer. Both the basis and the quantizer are fitted once, on a bootstrap shard, and read by every build job. A shard holds five million rows. A server holds 400 shards, two billion rows, on one block volume. The on-disk row is 12 bytes of codes, 8 bytes of per-row norms used by the asymmetric distance computation, and 4 bytes of cell membership. Every server reports 48 009 656 624 index bytes for its two billion rows, 24.0 bytes per row, and the fleet holds 24.0 TB of index on 500 volumes. Scoring is asymmetric distance computation over the codes, in row blocks, from memory-mapped shards. Because the basis and the coarse quantizer are shared, scores from different shards are comparable and a global top- k is the exact merge of per-shard top- k s. That property is what makes the measurement below exact rather than approximate, and Figure 1 shows the arrangement.

Corpus. The corpus is defined by a seed and regenerated on demand, so the distributed build moves no corpus bytes. Each global shard draws a fresh rank-16 Gaussian basis, projects Gaussian coefficients through it with a linear decay of singular values from 1.0 to 0.3, and adds five percent noise, in an ambient dimension of 32. Its intrinsic dimension measures 15.4 to 17.0 at every size we tested, with no trend in row count. Real embeddings such as Cohere Embed v3 at 1024 dimensions measure 52 to 61 by the same estimator. Hubness, the concentration of nearest-neighbour lists on a few points [13], rises with corpus size on this recipe, at about 0.42 per decade at $k = 10$ against roughly 0.25 per decade on real data. We attribute the rise to how shards are added. Each shard draws its own random 16-dimensional subspace, and two random 16-dimensional subspaces of a 32-dimensional space are not orthogonal, so the rows of a new shard land close to rows of shards already present. Every added shard therefore adds rows near existing ones,

and the rows that sit where many subspaces pass close together appear in more and more neighbour lists. Whatever its cause, the rise does not act on the quantity measured here. Recall against the exact scan depends only on whether the router reaches the cell that holds each reference neighbour (Section 4.2), and hubness changes which rows are the neighbours without changing that condition.

Build. The 500 servers were built between 2026-08-04 and 2026-09-24 by a pool of one job per server. A build job streams its 400 shards from the seed in row bands, writes each shard as it goes, and never holds a whole shard in memory, so it runs in the same one-CPU, two-gigabyte class as the measurement. Each server took between two and a half and four hours. Six servers whose volumes were lost to an offline node were deleted and rebuilt from seed in under four hours each, on the first attempt, which is the recovery path the seeded corpus makes available. The volumes are single-replica block volumes of 56 GiB, provisioned at a paced rate because a burst of requests times out the provisioner, and the shared volume that holds the bootstrap, the queries and every partial result is a network file system mounted by every job.

Scale points. The same protocol has been run at 10^8 rows on 4 servers, 10^9 on 4, 10^{10} on 8, 10^{11} on 50 and now 10^{12} on 500. Per-server work is constant from 10^{11} on, two billion rows, so the step from 50 to 500 servers changes nothing per node and changes only the number of chances to draw a slow or failing one.

3 PROTOCOL

The measurement has four phases, each a pool of independent jobs on the cluster’s batch system, every submission through a controller that paces submissions, caps concurrency at twenty and yields while the cluster is busy. The query-cache phase is one job that derives the query set, the first 25 rows of each of four seeded shards, 100 queries, and writes it to the shared volume. The queries are corpus rows by construction, so every server sees the same queries without regenerating anything. The reference phase is one job per server that scans that server’s two billion rows exactly with the asymmetric distance computation and writes its top-10 per query with scores. The routed phase is one job per server that runs the inverted-file search at 32 and at 128 probes and writes the same shape of partial. The score phase is one job that merges the 500 reference partials into the exact global top-10, merges the routed partials the same way, and reports recall at ten of the routed set against the reference set, per probe width, averaged over queries. A second routed phase at 16, 64 and 256 probes, run two days later, and three metadata phases that read only the partials and the index headers are described with their results in Sections 4.2 and 4.3.

Every job is idempotent on the volume. A job whose output already exists prints that fact and exits, and outputs are written to a temporary name and renamed, so a job interrupted mid-write leaves nothing behind. This is the property every recovery rule in Section 6 relies on. The score program has not changed since the 10^9 run. The two phase scripts changed at this run in the ways Section 5 describes, none of which touch what is computed.

Table 1: Recall at ten of routed search against the exact full scan of the same compressed index, by corpus size. Queries are 100 at 10^8 , 10^9 and 10^{12} rows and 500 at 10^{10} and 10^{11} . The last column is the median over servers of the wall time of one server’s exact scan. The 10^{12} jobs ran on one CPU each where the earlier runs had six, so that column is not comparable across rows (Section 4.4).

rows	servers	32 probes	128 probes	median scan (s)
10^8	4	0.979	0.999	47
10^9	4	see text		589
10^{10}	8	0.982	0.999	8495
10^{11}	50	0.983	0.999	19041
10^{12}	500	0.989	0.999	2664

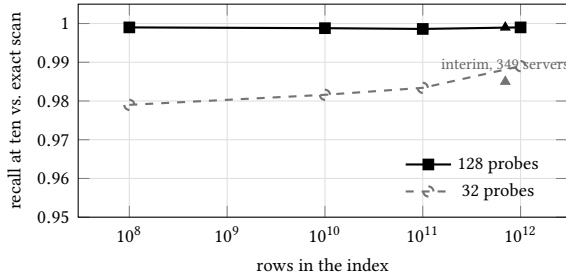


Figure 2: Recall against the exact scan by corpus size. The triangles are the interim score taken while the 10^{12} run was two thirds through its routed phase, on the 349 servers that had finished both phases, 698 billion rows. They sit on the final points.

4 RESULTS

4.1 Recall across four orders of magnitude

Table 1 and Figure 2 give the number the run exists to produce. At 10^{12} rows, routed search at 128 probes of 2048 cells returns 0.999 of the exact scan’s top ten, and at 32 probes 0.989. At 128 probes the figure is 0.999, 0.9988 and 0.9986 at the three smaller scales, a spread of 0.0004. At 32 probes it rises slowly with scale, 0.979, 0.982, 0.983, 0.989, which is consistent with the merge over more servers filling the tail of the shortlist and is not something we claim to explain. The interim score, taken on the first 349 servers to finish both phases, gave 0.985 and 0.999, and recall at 128 probes on the first half of those 349 servers alone was also 0.999. The number is stable inside the run as well as across runs.

The mean hides little. At 32 probes 90 of the 100 queries have recall 1.0, nine have 0.9 and one has 0.8. At 128 probes 99 queries have recall 1.0 and one has 0.9, so the mean of 0.999 is one query missing one neighbour. Resampling the 100 queries gives 95 percent bootstrap intervals of 0.982 to 0.995 at 32 probes and 0.997 to 1.000 at 128. The earlier scale points recorded only the mean, so they carry no interval.

The 10^9 row has no routed-versus-scan figure in the table because that run’s purpose was different. It carried a cold store of the original float vectors, so it measured true recall against float ground truth.

Table 2: Recall at ten predicted from the cell ranks of the reference neighbours, and recall measured by a routed pass at that width, 10^{12} rows, 100 queries. The 32 and 128 columns were measured first and checked against the prediction. The 16, 64 and 256 columns were predicted and recorded before the sweep that measured them was submitted. The last row counts the queries whose predicted recall is 1.0.

probes	1	2	4	8	16	32	64	128	256
predicted	0.406	0.599	0.727	0.867	0.952	0.989	0.997	0.999	1.000
measured					0.952	0.989	0.997	0.999	1.000
queries with recall 1.0	1	4	15	37	68	90	97	99	100

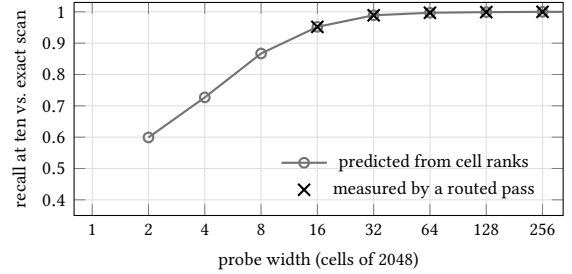


Figure 3: The recall curve over probe width computed from the cell rank of each reference neighbour (line, nine widths from the census merge, no distance scan) and the five widths measured by routed passes over the whole index (crosses). The crosses sit on the line at every width.

The routed shortlist at 128 probes returned 0.999 of the compressed scan there too, the compressed scan itself found only 0.592 of the true neighbours, and reranking the shortlist against the stored originals raised that to 0.991. That row is the reason Section 7 exists.

4.2 Recall from the probe order alone

Every reference neighbour lives in one of the 2048 cells, and each query orders the cells by the bound the router uses. Recall at width p is then the share of reference neighbours whose cell has rank below p in its query’s order, because inside a probed cell the routed pass scores with the same asymmetric distance the exact scan used, so a reference neighbour is returned exactly when its cell was probed. That identity, stated over finite sets with the in-cell condition as its hypothesis, is checked in Lean 4 with Mathlib in the repository (lean/Pvldb1t/Reachability.lean, theorem recall_eq_share). The share needs the cell of each of the 1000 reference neighbours and nothing else, so it was computed from the index metadata in 500 jobs of half a minute each and one merge, with no distance scan. Table 2 and Figure 3 give it at nine widths. It agreed with the two widths already measured, 32 and 128, to the last digit. We then recorded the values at 16, 64 and 256, submitted a routed pass at those three widths on all 500 servers, and the measured recall came back 0.952, 0.997 and 1.000, again equal to the prediction at every width. The neighbours’ cell ranks have median 1, ninetyeth percentile 9, ninety-ninth percentile 32 and maximum

169, so the single neighbour that 128 probes miss sits in the 170th cell of its query’s order, and 256 probes return every neighbour of every query.

The identity also explains the stability across scale. The corpus grows by adding shards under one shared coarse quantizer, so an added shard puts rows into the same 2048 cells and leaves every query’s probe order unchanged. What changes is which rows are the ten nearest, and recall at width p changes only if those rows move to cells of different rank. More rows in every cell put the nearest ten closer to the query, which is consistent with the slow rise at 32 probes. The agreement says that on this index recall is a property of the coarse quantizer’s probe order and of nothing downstream of it. It also gives a cheap instrument. Once the reference neighbours are known, the recall curve at any width is a count over cell ranks, and a routed pass at a new width confirms a number that was already on the page.

4.3 What the index metadata says

The same 500 metadata jobs took a census of the cells. The 2048 cells hold between 49.2 million and 2.15 billion rows, with mean 488 million and median 418 million, a Gini coefficient of 0.328, the largest one percent of cells holding 3.7 percent of rows, and no empty cell. The imbalance is the shared coarse quantizer’s, fitted once on the bootstrap shard, and it is what the router’s probe order works against.

The 1000 reference neighbours of the 100 queries sit on 233 of the 500 servers, and one server holds 16.6 percent of them, because the queries are rows of four seeded shards and each shard’s own basis keeps a query’s neighbours near its home shard. Losing one server at random would cost 0.002 of recall in expectation, and losing that one server would cost 0.166. The same recipe is used at every scale point, the first rows of four shards spread evenly across the index’s shard range, 25 rows each at 10^8 and 10^{12} rows and 125 each at 10^{10} and 10^{11} . The trend across sizes is therefore measured with the query distribution fixed. What the recipe does not show is the absolute recall for queries spread over many shards, whose neighbours would fall on more servers and into cells of other ranks. Measuring it takes a query set drawn from many shards and a new exact scan. The scan’s cost is set by the number of queries and not by how many shards they come from, so 100 queries taken two from each of 50 shards would cost the same 418 CPU-hours as this run, and 500 queries about five times as much.

4.4 Cost

Table 3 and Figure 4 give what the measurement cost. The exact scan of two billion rows at one CPU takes 44 minutes at the median and the routed pass 15 to 27 minutes depending on width. The first run, exact scan and routed passes at 32 and 128, consumed 772 CPU-hours, and the second sweep at 16, 64 and 256 consumed 577, so the whole measurement is 1349 CPU-hours of the cluster’s smallest job class. The routed times are not the cost of inverted-file search, and the ratio between the routed and exact rows is not a latency figure for the method. These jobs are read-bound on network block storage, spend a fixed minute or two on image pull, dependency install and clone, and open 400 shards before scoring. The 10^{11} run, on six CPUs per job, reported 14.7 and 8.5 as the

Table 3: Per-server wall time in the 10^{12} run, one CPU per job, 500 servers per phase, and the CPU-hours each phase consumed. The three routed widths of the second sweep ran inside one job per server and share its start-up. Each time includes pulling the image, installing dependencies, cloning the repository and opening 400 shards from network block storage, so it is the cost of the measurement job and not of inverted-file search. Derived from the record by `figdata/fig_data.py`.

phase	median	mean	p10	p90	max	CPU-h
exact scan	2664 s	3008 s	1850 s	4009 s	16308 s	418
routed, 32	1064 s	1131 s	775 s	1466 s	14286 s	157
routed, 128	1335 s	1420 s	958 s	1861 s	14524 s	197
routed, 16	916 s	1071 s	660 s	1440 s	12236 s	149
routed, 64	1219 s	1366 s	912 s	1743 s	14645 s	190
routed, 256	1594 s	1719 s	1176 s	2199 s	13786 s	239

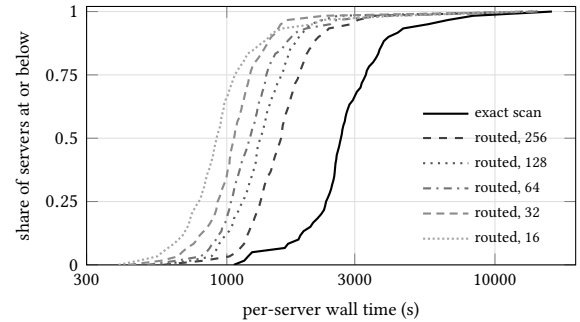


Figure 4: Distribution of per-server wall time by phase, 500 servers each, one CPU per job. The bodies are tight and the tails are a few servers on slow nodes, which the pool absorbs by waiting rather than by re-placing. Data derived from the record logs.

same ratio, and neither number is the method’s. The tails are node variance. Between 4 and 30 servers per phase took more than twice the median, the 16308 s maximum of the scan is one server on one node that also produced the slowest routed pass, and in every phase the ninetieth percentile is within 1.6 times the median.

5 ONE CPU AND TWO GIGABYTES PER JOB

Every job in the measurement requests one CPU and two gigabytes of memory. That is the smallest class the cluster offers, it is the class in which a fleet of hundreds of small jobs takes the least from other users, and the sharded index needs nothing more once three choices are made in how a scan over memory-mapped shards is written. The first is that the queries come from the cache. Deriving the 100 queries from the seed means regenerating four whole five-million-row blocks, about 640 MB each plus temporaries, so the query-cache phase writes the set once and every scan reads the file. The second is that at most two shards are open. A memory-mapped shard still materializes its row-id and tombstone arrays, 45 MB per five-million-row shard, and the sharded index holds 128 shards

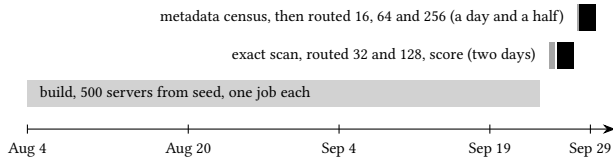


Figure 5: The run on the calendar. The index was built over seven weeks by a pool of one job per server. The measurement, the census and the second sweep took four days at the end of September, all through the same controller.

Table 4: The pool over the first run, 2026-09-25 18:01 UTC to 2026-09-27 08:59 UTC, and the second sweep, 2026-09-27 21:37 UTC to 2026-09-29 12:56 UTC. First-run scan counts exclude 152 servers scanned in an earlier session at a larger job shape.

	first run	second sweep
submissions	928	574
completions	849	501
recycles after a failed job	75	73
storage read fault on one node	24	25
node lost during the job	13	17
node unreachable, pod stuck	13	11
container start failure	2	1
other	23	19
recycles after 45 minutes pending	26	0
submissions deferred by the controller	1	0
servers parked or given up	0	0

open by default, 5.8 GB across a full scan. The scan holds two shards open and the routed pass eight, with two worker threads. The third is that scoring proceeds in blocks of 65536 rows, which bounds the per-block temporaries to tens of megabytes.

A pilot exact-scan job at this shape finished in 1912 s with CPU averaging 884 millicores and peaking at 979, memory averaging 1242 MiB and peaking at 1469, and a process resident set of 346 MB at the end. A pilot routed-pass job finished in 36 minutes at 475 millicores average, memory averaging 1258 MiB and peaking at 1568, resident set 165 MB. The memory above the resident set is clean page cache from the mapped index, which the kernel reclaims under the limit. The routed pod was observed at the two-gigabyte limit with 224 MB anonymous and 1.9 GB file-backed and completed normally. The score job, which merges 1500 small partials, runs in the same class, and so do the metadata jobs of Section 4.3.

6 RUNNING A POOL ON A SHARED CLUSTER

The measurement was driven by a pool of at most twenty jobs in flight with no wave barrier, checkpointed per phase, restartable, and able to adopt jobs already on the cluster after a restart. Figure 5 places it on the calendar and Table 4 gives what it did. The rules come first because the counts are only meaningful against them.

Rules. A job whose Kubernetes object reports success is done. A job whose object reports failure is recycled, meaning its object is deleted and, after an exponential back-off from two minutes to

thirty, resubmitted, up to forty times before the server is parked and swept again later. A job submitted through the controller but not yet visible as an object is waited on for ninety polls, one a minute, before it is treated as lost. In the first run a job whose pod had not reached the running state 45 minutes after submission was recycled the same way. For the second sweep that rule was replaced by waiting, because a running job is never deleted on a namespace other groups share, and a job running longer than sixteen hours is logged rather than removed.

The deferral rule. The submission controller defers creating a job while the namespace already has several pending or the cluster is busy, and retries for up to about an hour. A driver that treats a deferred submission as a lost one resubmits, and every resubmission queues a duplicate behind the original it was meant to replace. The wait before a submission is treated as lost separates the two cases. The run needed the rule once, for the score job.

Failure classes. Two dominated, and both are ordinary on a shared cluster. One node returned a storage read fault to every job placed on it within 3 to 13 minutes of start, after its block volume had attached. Retries landed elsewhere and finished, at a cost of about 130 slot-minutes out of 9600 over the first eight hours. The second class, pods pending past 45 minutes, was benign in every instance checked, because the original pod had run and the retry found its output within minutes. That is what idempotent jobs buy. In the second sweep 63 servers needed a second attempt, 9 a third and 1 a fourth, and a storage fault at one moment ended all twenty jobs in flight at once, after which fresh submissions ran and the batch was re-issued after its back-off.

Pods that stop answering. Three scan pods stayed in a terminating state on nodes that had become unreachable, and one routed pod ran at 34 millicores for 97 minutes on a node with a slow storage path. Removing the four pods by hand let the job controller replace them, and the replacements finished within an hour. The second sweep added a rule for pods terminating longer than thirty minutes and met one further case, a pod that could not mount its volume on one node for seventeen hours, which was removed by hand and re-placed. One server of the second sweep sat on the slow node for eleven hours and finished, holding the score for the last seven. Across four days the count of manual actions is six pod removals and one resubmission of the score job.

Provenance. The job containers cloned the repository’s default branch at run time, and twelve commits appear in pod logs across the first run. The phase scripts came from a configuration map fixed for each run, and the merge and score code did not change. Pinning the clone to a commit is the one change to the descriptor for the next run.

7 WHAT THE RESULT DOES AND DOES NOT SHOW

Recall of the routed shortlist against the exact scan of the same compressed index is flat from 10^8 to 10^{12} rows. That is a statement about the inverted-file layer, its shared coarse quantizer and its merge. The routing loses nothing the exact scan finds, at either probe width,

Table 5: Rerank bound on regenerated float rows, 100 queries, shortlists of width 100 cut from the merge of per-server top-10 partials. Survival is the share of the ADC top-10 in the float top-10 of the same shortlist, an upper bound on ADC-only true recall. Transfer is the share of the routed shortlist’s float top-10 that is in the reference shortlist’s float top-10.

shortlist	ref.	16	32	64	128	256
survival of the ADC top-10	0.621	0.623	0.623	0.624	0.622	0.621
transfer after rerank	–	0.940	0.982	0.996	1.000	1.000

when the corpus grows by adding shards, and Section 4.2 says why in one line.

It is not a statement about recall against the truth. The compressed scan at 10^9 rows found 0.592 of the float neighbours, and on a real corpus of 15 million Cohere Embed v3 vectors the same index found 0.693 of them by exact scan, 0.689 by routing at 256 probes, and 0.981 after reranking the routed shortlist against stored originals. On that real corpus the routed shortlist returned 0.990 of the compressed scan at 256 probes, so the routing result transfers, and the compression result does not. Four bits per dimension at this rate loses neighbours that only a rerank tier recovers, and the trillion-row corpus, at intrinsic dimension 16 against a real embedding’s 52 to 61, cannot show that. It is an easy corpus for the code and a hard one only for the hubness the shards’ overlap induces.

A cold store of originals at 10^{12} rows would be 128 TB of float vectors, so true recall and reranking remain established at 10^9 and at the 15-million-row real pilot, not here. The seeded corpus does allow one bound. The float rows named by a shortlist can be regenerated from the seed, so the ADC top-10 of each query can be reranked by float cosine within the shortlist that contains it, and the share of the ADC top-10 that survives that rerank is an upper bound on ADC-only true recall, because a corpus float neighbour that lies in the shortlist is a shortlist float neighbour. The set inequality behind the bound is checked in Lean 4 with Mathlib in the repository (Lean/Pvldb1t/RerankBound.lean, theorem rerank_upper_bound). Table 5 gives the result on 11002 regenerated rows. Of the reference scan’s ADC top-10, 0.621 survives the float rerank of its own shortlist on average, with 22 of 100 queries keeping all ten and a minimum of 0.100, so ADC-only true recall at 10^{12} rows is at most 0.621 on this corpus. The routed shortlists behave the same way, 0.623 at 16 probes and 0.621 at 256. At one billion rows the cold store gave ADC-only true recall of 0.592, inside this bound. After reranking, the routed top-10 agrees with the reranked reference top-10 at 0.940 for 16 probes, 0.982 for 32, 0.996 for 64 and 1.000 from 128 on, within 0.012 of the recall before reranking at every width and equal to it from 128 probes on, so the rerank neither hides nor amplifies what routing costs. The mean cosine of the ADC top-10 to the query is 0.863 and of the reranked top-10 0.929. Each query is itself a corpus row, and that row is in the ADC top-10 for 100 of 100 queries and in the reranked top-10 for 100 of 100.

One hundred queries is few. The exact scan is compute-bound in the query count and 500 queries would have cost about five times the 418 CPU-hours. And the queries come from four shards, so

Section 4.3’s concentration of reference neighbours on one server is a property of the query set rather than of the index.

8 RELATED WORK

Product quantization with asymmetric distance computation and inverted files [8] is the basis of the index, and the two-stage arrangement in which a compressed scan produces a shortlist that a finer code or the original vectors rerank dates from the same group’s billion-vector work [9]. Locally optimized codebooks per cell [11] are the nearest relative of the shared-basis arrangement we use to keep scores comparable across shards, and anisotropic quantization [6] and RaBitQ [4] are the current forms of the quantizer with, in the latter case, an error bound on the estimated distance. Locally adaptive scalar quantization [1] puts a compressed representation under a graph index and reranks from a second tier, which is the rerank tier our 10^9 and real-embedding figures use. Locality-sensitive hashing [7] and navigable graphs [12] are the usual alternatives to the inverted file and are not what this paper measures.

Billion-scale practice is established. Faiss [3, 10] on one machine or one GPU, DiskANN [16] and SPANN [2] on a single node with the index on disk, and the two Big ANN competitions [14, 15] that fixed the datasets and the reporting for that scale. Distributed vector databases such as Milvus [17] and its cloud-native successor [5] shard an index across nodes and merge per-node results, which is the arrangement our fleet uses without a serving layer, and recent work places a billion-scale index on distributed storage [19]. The literature the authors searched stops at 10^9 to 10^{10} rows for measured recall against an exact reference, and we found no published measurement of a routed index against the exact scan of the same index at 10^{12} rows. This paper adds that measurement, the identity that predicts it from the router’s probe order, and the design that obtained it on a shared cluster.

9 REPRODUCIBILITY

The repository at the availability URL carries the corpus seeds and generator, the build, scan, routed, score and metadata scripts, the pool driver, and under benchmarks/fleet/record/1t/ the pool state files, every driver log, the pilot logs, the interim score and the two score logs whose RESULT_JSON lines hold every per-server wall time. The figures and the cost table are produced from those logs by paper/pvldb1t/figdata/fig_data.py. The Lean development under paper/pvldb1t/lean/ builds against Mathlib at Lean 4.32.2, and its axiom audit, recorded in AXIOMS.txt, shows every theorem depending on propositional extensionality, quotient soundness and choice only. The twelve repository commits that job containers cloned during the first run are listed in the result note, and the configuration map that carried the phase scripts is recorded by content hash. Rebuilding the index from seed costs about three and a half hours per server at the pool’s width.

10 CONCLUSION

A trillion-row compressed index was built and measured exactly on a shared research cluster in the smallest resource class the platform offers, by jobs that read their queries from a cache, hold two shards open and score in blocks, submitted through a controller that yields

when the cluster is busy. On that index, routing loses nothing the exact scan finds, at 10^8 rows or at 10^{12} , and the recall at any probe width is the share of reference neighbours whose cell the router reaches, a count that predicted three measured widths to the last digit. Against float rows regenerated from the seed, the compressed ranking keeps at most 0.621 of the true neighbours, and a rerank of the routed shortlist recovers the reranked reference at 0.982 for 32 probes and 1.000 from 128 on. The measurement cost 1349 CPU-hours and four days, and it is reproducible from the repository, which carries the seeds, the scripts, the driver, the state files, the driver logs, the pilots, the interim score and the final scores with every per-server wall time.

ACKNOWLEDGMENTS

This work used the National Research Platform’s Nautilus cluster. The National Research Platform is supported in part by National Science Foundation awards CNS-1730158, ACI-1540112, ACI-1541349, OAC-1826967, OAC-2112167, CNS-2100237 and CNS-2120019. We thank its operators and community.

REFERENCES

- [1] Cecilia Aguerreberre, Ishwar Bhati, Mark Hildebrand, Mariano Tepper, and Ted Willke. 2023. Similarity Search in the Blink of an Eye with Compressed Indices. *Proceedings of the VLDB Endowment* 16, 11 (2023), 3433–3446. <https://doi.org/10.14778/3611479.3611537>
- [2] Qi Chen, Bing Zhao, Haidong Wang, Mingqin Li, Chuanjie Liu, Zengzhong Li, Mao Yang, and Jingdong Wang. 2021. SPANN: Highly-Efficient Billion-Scale Approximate Nearest Neighbor Search. In *Advances in Neural Information Processing Systems (NeurIPS)*.
- [3] Matthijs Douze, Alexandr Guzhva, Chengqi Deng, Jeff Johnson, Gergely Szilvasy, Pierre-Emmanuel Mazaré, Maria Lomeli, Lucas Hosseini, and Hervé Jégou. 2024. The Faiss Library. *arXiv preprint arXiv:2401.08281* (2024).
- [4] Jianyang Gao and Cheng Long. 2024. RaBitQ: Quantizing High-Dimensional Vectors with a Theoretical Error Bound for Approximate Nearest Neighbor Search. *Proceedings of the ACM on Management of Data* 2, 3 (2024). <https://doi.org/10.1145/3654970>
- [5] Rentong Guo, Xiaofan Luan, Long Xiang, Xiao Yan, Xiaomeng Yi, Jigao Luo, Qianya Cheng, Weizhi Xu, Jiarui Luo, Frank Liu, Zhenshan Cao, Yanliang Qiao, Ting Wang, Bo Tang, and Charles Xie. 2022. Manu: A Cloud Native Vector Database Management System. *Proceedings of the VLDB Endowment* 15, 12 (2022), 3548–3561. <https://doi.org/10.14778/3554821.3554843>
- [6] Ruiqi Guo, Philip Sun, Erik Lindgren, Quan Geng, David Simcha, Felix Chern, and Sanjiv Kumar. 2020. Accelerating Large-Scale Inference with Anisotropic Vector Quantization. In *Proceedings of the 37th International Conference on Machine Learning (ICML) (Proceedings of Machine Learning Research)*, Vol. 119. 3887–3896.
- [7] Piotr Indyk and Rajeev Motwani. 1998. Approximate Nearest Neighbors: Towards Removing the Curse of Dimensionality. In *Proceedings of the 30th Annual ACM Symposium on Theory of Computing (STOC)*. 604–613.
- [8] Hervé Jégou, Matthijs Douze, and Cordelia Schmid. 2011. Product Quantization for Nearest Neighbor Search. *IEEE Transactions on Pattern Analysis and Machine Intelligence* 33, 1 (2011), 117–128.
- [9] Hervé Jégou, Romain Tavenard, Matthijs Douze, and Laurent Amsaleg. 2011. Searching in One Billion Vectors: Re-rank with Source Coding. In *IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*. 861–864. <https://doi.org/10.1109/ICASSP.2011.5946540>
- [10] Jeff Johnson, Matthijs Douze, and Hervé Jégou. 2021. Billion-Scale Similarity Search with GPUs. *IEEE Transactions on Big Data* 7, 3 (2021), 535–547.
- [11] Yannis Kalantidis and Yannis Avrithis. 2014. Locally Optimized Product Quantization for Approximate Nearest Neighbor Search. In *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. 2321–2328.
- [12] Yury A. Malkov and Dmitry A. Yashunin. 2020. Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs. *IEEE Transactions on Pattern Analysis and Machine Intelligence* 42, 4 (2020), 824–836.
- [13] Miloš Radovanović, Alexandros Nanopoulos, and Mirjana Ivanović. 2010. Hubs in Space: Popular Nearest Neighbors in High-Dimensional Data. *Journal of Machine Learning Research* 11 (2010), 2487–2531.
- [14] Harsha Vardhan Simhadri, Martin Aumüller, Amir Ingber, Matthijs Douze, George Williams, Magdalen Dobson Manohar, Dmitry Baranchuk, Edo Liberty, Frank Liu, Ben Landrum, Mazin Karjkar, Laxman Dhulipala, Meng Chen, Yue Chen, Rui Ma, Kai Zhang, Yuzheng Cai, Jiayang Shi, Yizhuo Chen, Weiguo Zheng, Zihao Wan, Jie Yin, and Ben Huang. 2024. Results of the Big ANN: NeurIPS’23 competition. *arXiv preprint arXiv:2409.17424* (2024).
- [15] Harsha Vardhan Simhadri, George Williams, Martin Aumüller, Matthijs Douze, Artem Babenko, Dmitry Baranchuk, Qi Chen, Lucas Hosseini, Ravishankar Krishnaswamy, Gopal Srinivasa, Suhas Jayaram Subramanya, and Jingdong Wang. 2022. Results of the NeurIPS’21 Challenge on Billion-Scale Approximate Nearest Neighbor Search. In *Proceedings of the NeurIPS 2021 Competitions and Demonstrations Track (Proceedings of Machine Learning Research)*, Vol. 176. 177–189.
- [16] Suhas Jayaram Subramanya, Devvrit, Rohan Kadekodi, Ravishankar Krishnaswamy, and Harsha Vardhan Simhadri. 2019. DiskANN: Fast Accurate Billion-point Nearest Neighbor Search on a Single Node. In *Advances in Neural Information Processing Systems*, Vol. 32.
- [17] Jianguo Wang, Xiaomeng Yi, Rentong Guo, Hai Jin, Peng Xu, Shengjun Li, Xiangyu Wang, Xiangzhou Guo, Chengming Li, Xiaohai Xu, Kun Yu, Yuxing Yuan, Yinghao Zou, Jiquan Long, Yudong Cai, Zhenxiang Li, Zhifeng Zhang, Yihua Mo, Jun Gu, Ruiyi Jiang, Yi Wei, and Charles Xie. 2021. Milvus: A Purpose-Built Vector Data Management System. In *Proceedings of the 2021 International Conference on Management of Data (SIGMOD)*. 2614–2627. <https://doi.org/10.1145/3448016.3457550>
- [18] Derek Weitzel, Ashton Graves, Sam Albin, Huijun Zhu, Frank Würthwein, Mahidhar Tatineni, Dmitry Mishin, John Graham, Elham E. Khoda, Mohammad Firas Sada, Larry Smarr, and Thomas DeFanti. 2025. The National Research Platform: Stretched, Multi-Tenant, Scientific Kubernetes Cluster. In *Practice and Experience in Advanced Research Computing (PEARC ’25)*. arXiv:2505.22864.
- [19] Kun Yu, Jiabao Jin, Xiaoyao Zhong, Peng Cheng, Lei Chen, Zhitao Shen, Jingkuan Song, Hengtao Shen, and Xuemin Lin. 2025. Approximate Nearest Neighbor Search of Large Scale Vectors on Distributed Storage. *arXiv preprint arXiv:2510.17326* (2025).